Evidence 2 - Generating and Cleaning a Restricted Context Free Grammar

Moritz Trieu

May 2026

1 Introduction

Languages can be categorized into different categories, for example through the Chomsky hierarchy. Description can be found by [Geeks-For-Geeks, 2026b]. The language in this evidence is set out to be parsed with an LL(1) parser which is a top-down parser without backtracking as explained in [Wikipedia-Contributors, 2026a].

2 Language

The language chosen was the Prolog programming language as for example described in [Wikipedia-Contributors, 2026b]. In comparison to other languages, Prolog does not become incredibly complex. Still there are quite a few aspects to be considered. Therefore, to keep the scope contained, only features used during the lecture were considered. These are predicates, clauses, lists and the cut operator, which are also explained in [Loiseleur and Vigier., 2001].

2.1 Structure and Terminals

While not necessary in Prolog itself, here the restriction is made that all predicates have to come before any clause. As this is usually done in practice, this doesn't display a big restriction and simplifies the grammar going forward. A predicate consists of a name, braces, possible arguments and an ending ".". Since the name of the predicate can consist of a string that follows certain rules but is otherwise ambiguous, defining a vocabulary as with other languages seems futile. Instead, the name can be categorized by lexical analysis; therefore, the decision was made to only have one categorical terminal γ which represents all valid predicate names. The same goes for the attributes, which are represented by the terminal α. More than a name or a string an attribute can also be a list. Lists can be represented in two ways, either fully written out like "[a, b, c, d]" or as a head-tail representation "[H-T]". Note that "-" was substituted for "|" to avoid notational confusion later on.

(a) Expanding from the right (b) Expanding from the left

Figure 1: Three attributes can be generated in more then one way.

Finally, a clause consists of a name, attributes, the symbol ":-" and a string of an ambiguous number of predicates and cut operators, delimited by commas or semicolons.

2.2 Initial Grammar

This description leads to the first grammar model.

E –> P C
P –> P γ (A) | ϵ
A –> A, A | α | [T]
T –> [T] T'' | α T' | ϵ
T' –> – A' | , T | ϵ
T'' –> , T | ϵ
C –> μ (W) :– B . C | ϵ
W –> V | ϵ
V –> V , V | ν
B –> B , B | B ; B | γ (A) | !

Since we want this grammar to be compatible with an LL(1) parser, we need to analyze for ambiguity and left recursion. As LL(1) doesn't use backtracking, the grammar needs to be deterministic as explained in [Wikipedia-Contributors, 2026a]. We can see for example that the generation of attributes is ambiguous. The attribute series "α, α, α" can be generated in two ways as can be seen in Fig. 1. The same can be done for

V –> V,V | ν
B –> B,B | B;B | γ(I) | !

.

2.3 Eliminating Ambiguity

To eliminate ambiguity, an extra step needs to be added as explained in [Geeks-For-Geeks, 2026a]. Thus, the aforementioned rules become

E –> P C

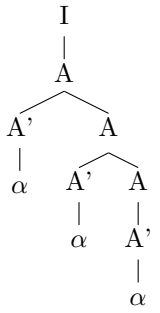

Figure 2: Building α, α, α using the updated grammar

```
P –> P γ ( A ) | ϵ
A –>  A, A'
A' –> α | [ T ]
T –> [ T ] T'' | α T' | ϵ
T' –> – A' | , T | ϵ
T'' –> , T | ϵ
C –>  μ ( W ) :– B . C | ϵ
W –> V | ϵ
V –> V, V'
V' –> ν
B –> B , B'
B' –> B ' ; B'''
B''' –> γ ( A ) | !
```

If we now try to rebuild "α, α, α", we can see in Fig. 2 it can now only be build in one way.

3 Eliminate Left Recursion

Left recursion appears whenever a production calls itself in the leftmost side as we can observe for example for

```
P –> P γ(I). | ϵ
```

As the string is parsed from left to right in LL(1), the tree must not extend indefinetly to the left side. To eliminate left recursion, as explained in [Maza, 2004], the production is substituted with another production that starts with a terminal and goes into a intermediate step that can result into empty.

```
P –> P'
P' –> γ(I).P' | ϵ
```

This results in a tree as can be seen in Fig. 3.

(a) A grammar rule that contains left-left-recursion. The tree can indefinetly expand to the left.

(b) Changes grammar to right-recursion. Now the tree grows to the right.

Figure 3: Changing the left-recursion to right recursion by adding an intermediate step.

3.1 Final Grammar

With all these changes we get to the final grammar rules:

E –> P C
P –> P'
P' –> γ (A) . P' | ϵ
A –> A' A'' | ϵ
A'' –> , A' A'' | ϵ
A' –> α | [T]
T –> [T] T' | α T' | ϵ
T' –> – A' | , T | ϵ
T'' –> , T | ϵ
C –> μ (W) :– B . C | ϵ
W –> V | ϵ
V –> V' V''
V'' –> , V' V'' | ϵ
V' –> ν
B –> B' B''
B'' –> , B' B'' | ϵ
B' –> B''' B''''
B'''' –> ; B''' B'''' | ϵ
B''' –> γ (A) | !

4 Implementation and Testing

Using the `nltk` python library from [NLTK-Project, 2025] a series of test sentences were parsed with LL(1) using the cleaned grammar. The implementation can be found in `parser.ipynb`. The test sentences are parted into two subsets, sentences the grammar should accept and sentences that should be rejected. For the accepted sentences, the parser returns the syntax-trees. Since ambiguity has been removed, there needs to be exactly one tree possible.

The accepted sentences set is

1. gamma ( ) .
2. gamma (alpha) .
3. gamma (alpha , alpha , alpha) .
4. gamma ([ ] , alpha) .
5. gamma (alpha , [ ]) .
6. gamma (alpha , [[ ]]) .
7. gamma (alpha , [alpha - alpha]) .
8. gamma (alpha , [alpha - [alpha]]) .
9. gamma (alpha , [alpha , alpha , alpha]) .
10. gamma (alpha , [alpha , [alpha , alpha] , alpha]) .
11. gamma (alpha) . gamma (alpha) .
12. mu (nu , nu) :- gamma (alpha) , gamma (alpha) . mu ( ) :- gamma ( ) ; gamma ( ) , ! .

Sentences 1. - 3. & 11 test predicate grammar, sentences 4. - 10. test lists and sentence 12 tests clause grammar.

The sentences that should be rejected are

1. gamma alpha
2. gamma (alpha alpha) .
3. gamma (alpha , alpha)
4. gamma (alpha , [[alpha] - alpha]) .
5. mu (nu , nu) :- gamma (alpha)
6. mu (nu nu) :- gamma (alpha) .
7. mu (nu , nu) :- gamma (alpha) mu (nu , nu) :- gamma (alpha)
8. gamma (alpha , alpha) mu (nu , nu) :- gamma (alpha) .

and include various errors such as missing terminals or wrong construction of lists.

5 Analysis

Due to its nature, the language from the begining is set out to be a context-free language. A push-down automaton is required to keep track of the sentence generation, thus it is above a regular language. Since the cleaned grammar is designed to be parsed by LL(1) it is also a context-free grammar instead of a grammar-sensitive language. When looking at the description of a context sensitive language as in [Geeks-For-Geeks, 2026c], the initial grammar doesn't utilize any context clues, thus both the initial and cleaned grammar are context-free languages, which makes intuetivly sense given the programatic nature of the language grammar.

References

[Geeks-For-Geeks, 2026a] Geeks-For-Geeks (2026a). Ambiguous grammar.

[Geeks-For-Geeks, 2026b] Geeks-For-Geeks (2026b). Chomsky hierarchy in theory of computation.

[Geeks-For-Geeks, 2026c] Geeks-For-Geeks (2026c). Context-sensitive grammar (csg) and language (csl).

[Loiseleur and Vigier., 2001] Loiseleur, M. and Vigier., N. (2001). Prolog.

[Maza, 2004] Maza, M. M. (2004). Elimination of left recursion.

[NLTK-Project, 2025] NLTK-Project (2025). Nltk.

[Wikipedia-Contributors, 2026a] Wikipedia-Contributors (2026a). Ll parser — Wikipedia, the free encyclopedia. [Online; accessed 4-June-2026].

[Wikipedia-Contributors, 2026b] Wikipedia-Contributors (2026b). Prolog — Wikipedia, the free encyclopedia. [Online; accessed 4-June-2026].